

In [29]: *# 필요한 라이브러리 불러오기.*

```
import os
import sys
import torch
import wandb

from datetime import datetime
from random import sample
from torch import nn, optim
from torch.utils.data import DataLoader, random_split, ConcatDataset
from torchvision import datasets
from torchvision.transforms import transforms, v2
from torchinfo import summary

BASE_PATH = '/home/ksy/Develop/Kut-Deep-Learning-250204'
BASE_PATH = 'C:\\Users\\first\\Develop\\Kut-Deep-Learning-250204'
sys.path.append(BASE_PATH)

from _01_code._99_common_utils.utils import get_num_cpu_cores
from _01_code._09_fcn_best_practice.c_trainer import ClassificationTrainer
from _01_code._09_fcn_best_practice.d_tester import ClassificationTester

CURRENT_FILE_PATH = f'{BASE_PATH}/_03_homeworks/homework_3'
CHECKPOINT_FILE_PATH = os.path.join(CURRENT_FILE_PATH, "checkpoints")
if not os.path.isdir(CHECKPOINT_FILE_PATH):
    os.makedirs(os.path.join(CURRENT_FILE_PATH, "checkpoints"))
```

In [30]: **def** get_fashion_mnist_data():

```
    data_path = os.path.join(BASE_PATH, "_00_data", "j_fashion_mnist")

    f_mnist_train = datasets.FashionMNIST(data_path, train=True, download=True,
    f_mnist_train, f_mnist_validation = random_split(f_mnist_train, [55_000, 5_0

    if True:
        # 0/1/2 Augmentation.
        img_transform = v2.Compose([
            v2.RandomHorizontalFlip(),
            v2.RandomCrop([28, 28], padding=4),
        ])

        transformed_f_mnist_train = []
        for img, label in f_mnist_train:
            timg = img_transform(img)
            transformed_f_mnist_train.append((timg, label))
        f_mnist_train = ConcatDataset([f_mnist_train, transformed_f_mnist_train]

    # mean std
    if True:
        # 이미지 Augmentation을 진행했으므로 자동으로 mean, std를 계산할 수 있도록
        train_imgs = torch.stack([i for i, _ in f_mnist_train], dim=3)
        train_stat = (train_imgs.view(1, -1).mean(), train_imgs.view(1, -1).std()
        print(f'f_mnist_train:')
        print(f'    mean: {train_stat[0]}')
        print(f'    std: {train_stat[1]}')
    else:
        # 원본 데이터의 mean과 std.
        # f_mnist_train:
        #    mean: 0.28632092475891113
```

```

        #      std: 0.353121280670166
        train_stat = (0.28632092475891113, 0.353121280670166)

    print("Num Train Samples: ", len(f_mnist_train))
    print("Num Validation Samples: ", len(f_mnist_validation))
    print(f"Stat Train Samples: (mean, std): {train_stat}")
    print("Sample Data Shape: ", f_mnist_train[0][0].shape) # torch.Size([1, 28
    print("Sample Data Target: ", f_mnist_train[0][1]) # 9

    num_data_loading_workers = get_num_cpu_cores()
    print("Number of Data Loading Workers:", num_data_loading_workers)

    train_data_loader = DataLoader(
        dataset=f_mnist_train, batch_size=wandb.config.batch_size, shuffle=True,
        pin_memory=True, num_workers=num_data_loading_workers
    )

    validation_data_loader = DataLoader(
        dataset=f_mnist_validation, batch_size=wandb.config.batch_size,
        pin_memory=True, num_workers=num_data_loading_workers
    )

    f_mnist_transforms = nn.Sequential(
        transforms.ConvertImageDtype(torch.float),
        transforms.Normalize(mean=train_stat[0], std=train_stat[1]),
    )

    return train_data_loader, validation_data_loader, f_mnist_transforms

```

```

In [31]: def get_fashion_mnist_test_data():
    data_path = os.path.join(BASE_PATH, "_00_data", "j_fashion_mnist")

    f_mnist_test_images = datasets.FashionMNIST(data_path, train=False, download
    f_mnist_test = datasets.FashionMNIST(data_path, train=False, download=True,

    # mean std
    if False:
        test_imgs = torch.stack([i for i, _ in f_mnist_test], dim=3)
        print(f'f_mnist_test:')
        print(f'    mean: {test_imgs.view(1, -1).mean()}')
        print(f'    std: {test_imgs.view(1, -1).std()}')

        print('[INFO] Calculating mean, std done. quitting...')
        exit(0)
    else:
        # f_mnist_train:
        #      mean: 0.2868492901325226
        #      std: 0.3524441719055176
        test_stat = (0.2868492901325226, 0.3524441719055176)

    print("Num Test Samples: ", len(f_mnist_test))
    print(f"Stat Test Samples: (mean, std): {test_stat}")
    print("Sample Shape: ", f_mnist_test[0][0].shape) # torch.Size([1, 28, 28])

    test_data_loader = DataLoader(dataset=f_mnist_test, batch_size=len(f_mnist_t

    f_mnist_transforms = nn.Sequential(
        transforms.ConvertImageDtype(torch.float),
        transforms.Normalize(mean=test_stat[0], std=test_stat[1]),
    )

```

```
return f_mnist_test_images, test_data_loader, f_mnist_transforms
```

```
In [32]: def get_cnn_model():
class MyModel(nn.Module):
    def __init__(self, in_channels, n_output):
        super().__init__()

        self.model = nn.Sequential(
            # B x 1 x 28 x 28 --> B x 6 x (28 - 5 + 1) x (28 - 5 + 1) = B x 6 x 24 x 24
            nn.Conv2d(in_channels=in_channels, out_channels=6, kernel_size=(5, 5), stride=(1, 1)),
            # B x 6 x 24 x 24 --> B x 6 x 12 x 12
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.ReLU(),
            # B x 6 x 12 x 12 --> B x 16 x (12 - 5 + 1) x (12 - 5 + 1) = B x 16 x 8 x 8
            nn.Conv2d(in_channels=6, out_channels=16, kernel_size=(5, 5), stride=(1, 1)),
            # B x 16 x 8 x 8 --> B x 16 x 4 x 4
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.ReLU(),
            # B x 16 x 4 x 4 --> B x 256
            nn.Flatten(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, n_output),
        )

    def forward(self, x):
        x = self.model(x)
        return x

# 1 * 28 * 28
my_model = MyModel(in_channels=1, n_output=10)

return my_model
```

```
In [33]: def train(args):
    run_time_str = datetime.now().astimezone().strftime('%Y-%m-%d_%H-%M-%S')

    config = {
        'epochs': args.epochs,
        'batch_size': args.batch_size,
        'validation_intervals': args.validation_intervals,
        'learning_rate': args.learning_rate,
        'early_stop_patience': args.early_stop_patience,
        'early_stop_delta': args.early_stop_delta
    }

    project_name = "cnn_fashion_mnist"
    project_name = args.project_name

    device = args.device
    print(f"Training on device {device}.")

    wandb.init(
        mode="online" if args.wandb else "disabled",
        project=project_name,
        notes="mnist experiment with cnn",
        tags=["cnn", "mnist"],
        name=run_time_str,
```

```

        config=config
    )
    print(args)
    print(wandb.config)

    train_data_loader, validation_data_loader, f_mnist_transforms = get_fashion_mnist_data_loader(
        model = get_cnn_model()
        model.to(device)

    summary(model=model, device=device, input_size=(1, 1, 28, 28))

    optimizer = optim.SGD(model.parameters(), lr=wandb.config.learning_rate)

    classification_trainer = ClassificationTrainer(
        project_name, model, optimizer, train_data_loader, validation_data_loader, f_mnist_transforms,
        run_time_str, wandb, device, CHECKPOINT_FILE_PATH
    )
    classification_trainer.train_loop()

    wandb.finish()

```

```

In [38]: def test(args):
    model = get_cnn_model()

    summary_model = get_cnn_model()
    summary_model.to(args.device)
    summary(model=summary_model, device=args.device, input_size=(1, 1, 28, 28))
    print('wtf?')

    test_imgs, test_data_loader, f_mnist_test_transforms = get_fashion_mnist_test_data_loader(
        classification_tester = ClassificationTester(
            args.project_name, model, test_data_loader, f_mnist_test_transforms, CHECKPOINT_FILE_PATH
        )

    for imgs, targets in test_data_loader:
        combined = list(zip([img for img, _ in test_imgs], imgs, targets))

        while True:
            xs = []
            ys = []
            zs = []

            cases = sample(combined, 10)
            for img, img_tensor, target in cases:
                img_tensor = img_tensor.unsqueeze(dim=1)
                pred = classification_tester.test_single(img_tensor)

                xs.append(pred)
                ys.append(int(target))
                zs.append(img)

            if any(x != y for x, y in zip(xs, ys)):
                print(xs)
                print(ys)
                print(zs)
                break

        for x, y, z in zip(xs, ys, zs):
            labels = {
                0: '티셔츠/상의 (T-shirt/top)',

```

```

1: '바지 (Trouser)',
2: '풀오버 (Pullover)',
3: '드레스 (Dress)',
4: '코트 (Coat)',
5: '샌들 (Sandal)',
6: '셔츠 (Shirt)',
7: '스니커즈 (Sneaker)',
8: '가방 (Bag)',
9: '발목 부츠 (Ankle boot)',
}

if x != y:
    print(f'Predicted: {x} - {labels[x]}, Ground Truth: {y} - {labels[y]}')
    import matplotlib.pyplot as plt
    plt.imshow(z, cmap='gray')
    plt.show()
    break

```

```

In [41]: # dict에 접근할 때 `.` 연산자로 접근할 수 있도록 하는 dict.
# stolen from: https://stackoverflow.com/questions/2352181/how-to-use-a-dot-to-a
class dotdict(dict):
    """dot.notation access to dictionary attributes"""
    __getattr__ = dict.get
    __setattr__ = dict.__setitem__
    __delattr__ = dict.__delitem__

args = dotdict({
    'wandb': True,
    'epochs': 10000,
    'batch_size': 512,
    'validation_intervals': 10,
    'learning_rate': 0.001,
    'early_stop_patience': 10,
    'early_stop_delta': 1e-5,
    'project_name': 'cnn_fashion_mnist',
    'device': torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
})

# train(args)
test(args)
print('done')

# wandb: https://wandb.ai/unta-korea-university-of-technology-and-education/cnn\_

```

wtf?

Num Test Samples: 10000

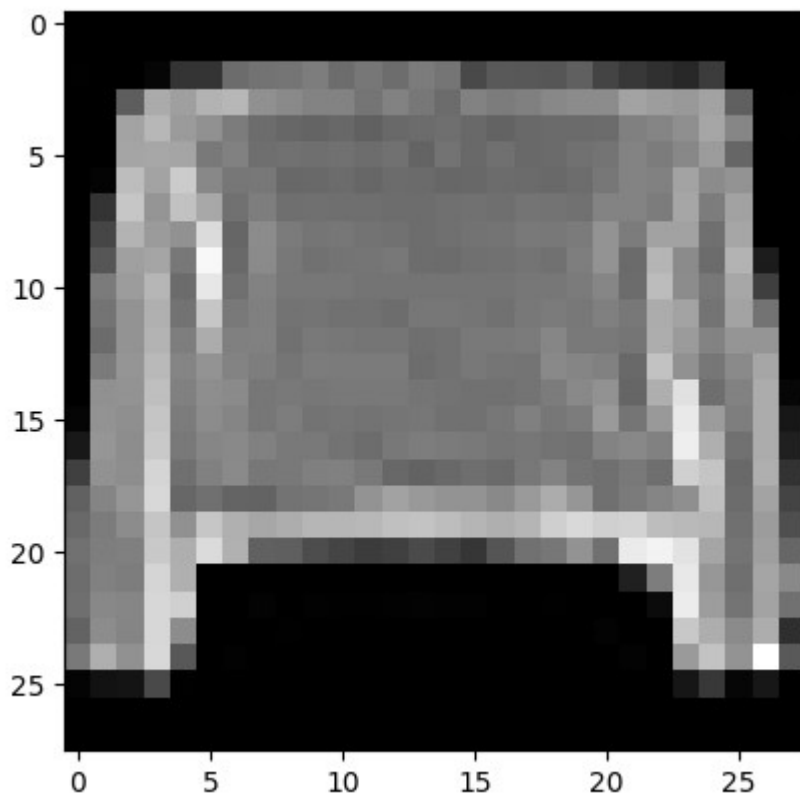
Stat Test Samples: (mean, std): (0.2868492901325226, 0.3524441719055176)

Sample Shape: torch.Size([1, 28, 28])

MODEL FILE: C:\Users\first\Develop\Kut-Deep-Learning-250204_03_homeworks/homework_3\checkpoints\cnn_fashion_mnist_checkpoint_latest.pt

```
C:\Users\first\Develop\Kut-Deep-Learning-250204\_01_code\_09_fcn_best_practice\d_
tester.py:18: FutureWarning: You are using `torch.load` with `weights_only=False`
(the current default value), which uses the default pickle module implicitly. It
is possible to construct malicious pickle data which will execute arbitrary code
during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#u
ntrusted-models for more details). In a future release, the default value for `we
ights_only` will be flipped to `True`. This limits the functions that could be ex
ecuted during unpickling. Arbitrary objects will no longer be allowed to be loade
d via this mode unless they are explicitly allowlisted by the user via `torch.ser
ialization.add_safe_globals`. We recommend you start setting `weights_only=True`
for any use case where you don't have full control of the loaded file. Please ope
n an issue on GitHub for any issues related to this experimental feature.
```

```
self.model.load_state_dict(torch.load(self.latest_file_path, map_location=torch
h.device('cpu'))))
[6, 8, 7, 5, 0, 2, 0, 8, 7, 9]
[6, 8, 7, 5, 0, 2, 0, 6, 7, 9]
[<PIL.Image.Image image mode=L size=28x28 at 0x1E6ABF82C30>, <PIL.Image.Image ima
ge mode=L size=28x28 at 0x1E6976C6510>, <PIL.Image.Image image mode=L size=28x28
at 0x1E6BAC36060>, <PIL.Image.Image image mode=L size=28x28 at 0x1E6ABA52390>, <P
IL.Image.Image image mode=L size=28x28 at 0x1E6BACF1D0>, <PIL.Image.Image image
mode=L size=28x28 at 0x1E6BAC5B0E0>, <PIL.Image.Image image mode=L size=28x28 at
0x1E697605850>, <PIL.Image.Image image mode=L size=28x28 at 0x1E6ABAE1B0>, <PIL.
Image.Image image mode=L size=28x28 at 0x1E6BAC37110>, <PIL.Image.Image image mod
e=L size=28x28 at 0x1E69768E750>]
Predicted: 8 - 가방 (Bag), Ground Truth: 6 - 셔츠 (Shirt)
```



done

숙제 후기

먼저 방대한 양의 데이터를 처음 학습/검증 및 테스트 해봤는데 생각보다 시간이 더 오래 걸려서 놀랐다.

또 로스에 개선이 없을 때 학습을 중단할 수 있도록 했는데 계속 중간에 학습이 중단돼

서 아쉬웠다.

한편 입력으로 주어지는 데이터는 보통의 MNIST 데이터셋과 동일하게 흑백의 28*28 해상도 이미지였는데, 생각보다 옷의 종류를 잘 구분했다. 다만, 해상도의 한계로 전체적인 외형이 비슷하고 세부적인 것으로 종류를 구분해야 하는 경우에는 잘 구분하지 못했다.

결과에 대한 간단한 고찰

최적의 하이퍼파라미터

이번 과제는 학교 외적인 일로 바쁜 일이 많아 여러 가지 조건에서 다양한 테스트를 하지는 못했다. 그렇기에 간단하게 이미지 Augmentation과 배치 사이즈 조절 등으로 정확도를 올려봤다.

이미지 Augmentation

먼저 이미지 Augmentation은 학습 셋의 이미지를 약간 변형하여 추가적인 데이터를 확보하는 것으로 기존의 데이터를 변형하는 것에서 그저 중복된 데이터를 추가하는 게 아닌가 하는 의문이 들었다.

하지만 이미지를 변형함으로써 다양한 시각에서 바라본 물체에 대한 정보를 제공할 수 있다.

예를 들어 어떤 물체를 정면으로 바라봤을 때와 정면에서 벗어나서 바라본 모습을 제공할 수 있다.

또 이미지를 기울이는 등의 왜곡을 주어 사물이 거울이나 카메라의 왜곡이 생긴 경우 등 다양한 변인에 대처할 수 있도록 할 수 있다.

그 결과 검증 로스가 각 에포크마다 조금씩 더 줄어드는 효과를 볼 수 있었다. 그러나 소극적인 변형 탓인지 극적인 효과를 보지는 못했다.

배치 사이즈

배치 사이즈는 학습 시 데이터를 얼마큼씩 볼 건지를 조정하는 하이퍼파라미터로 볼 수 있다.

너무 작다면 지엽적인 학습이 될 수 있고 너무 크다면 세부적인 디테일을 놓칠 수 있다.

다만 통상적으로 간단한 구조의 모델에서는 작은 배치사이즈가, 복잡한 구조의 모델에서는 큰 배치사이즈가 성능이 좋다고 알려져 있다.

최초 2048의 배치 사이즈에서 512까지 크기를 줄여가며 학습해보니 512에서 가장 높은 성능을 보였다.

다만 데이터나 모델 구조 자체를 변형한 것은 아니라서 로스가 줄어드는 속도에만 영향이 있었고 절대적인 로스를 더 낮춰주지는 않았다.

테스트 데이터 분류 예측 결과

위에서 셔츠를 가방으로 잘못 예측한 사진이 있는데 이는 상기한 것처럼 해상도나 색상의 부족으로 나타나는 결과로 생각된다.

먼저 셔츠와 가방은 용도부터 다르므로 사용되는 색상 등이 다를 수 있다. 셔츠는 밝은 계열 색상이 많을 수 있고 가방은 어두운 계열 색상이 많을 수 있다. 그런데 색상 정보가 없으므로 이를 통해 종류를 예측하기 어렵다.

다음으로는 해상도인데 두 의류 모두 넓은 면과 같은 부분이 있을 수 있는 종류인데 사진에서 이와 같은 넓은 공간이 관측된다.

이러한 복합적인 이유로 잘못된 예측을 하지 않았을까 한다.